

14

REINFORCEMENT LEARNING & TRANSFORMER MODELS: TRANSFORMER-BASED ONLINE RL

ДАНИИЛ
ЗЕЛЕЗЕЦКИЙ



НИКИТА
КАЧАЕВ



12 ДЕКАБРЯ
17:00



Center for
Cognitive
Modeling



Cognitive
Dynamic
Systems



ITL

Intelligent
Transport
Lab



ИНСТИТУТ
ИСКУССТВЕННОГО
ИНТЕЛЛЕКТА
МФТИ

ФПМИ



МФТИ

Outline:

- Key Advantages of Transformer-based RL
- Ways to train Transformer and their corresponding difficulties
- TOP-ERL: Off-Policy Online RL Algorithm
- Current results in Off-Policy Transformer RL

Key Advantages of Transformers in RL

Attention mechanism: Enables the RL agent to focus selectively on relevant cues in the environment and ignore redundant features, which leads to faster training. This is particularly useful in high-dimensional state spaces where there are a large number of input elements.

Multi-Modal Architectures: For complex tasks, RL agents may require additional information from different data modalities. Transformers can process multiple modalities of data (e.g., text, images) effectively using the same architecture.

Parallel Processing: Transformers are well-suited for parallelization due to their self-attention mechanism, which considers all inputs simultaneously.

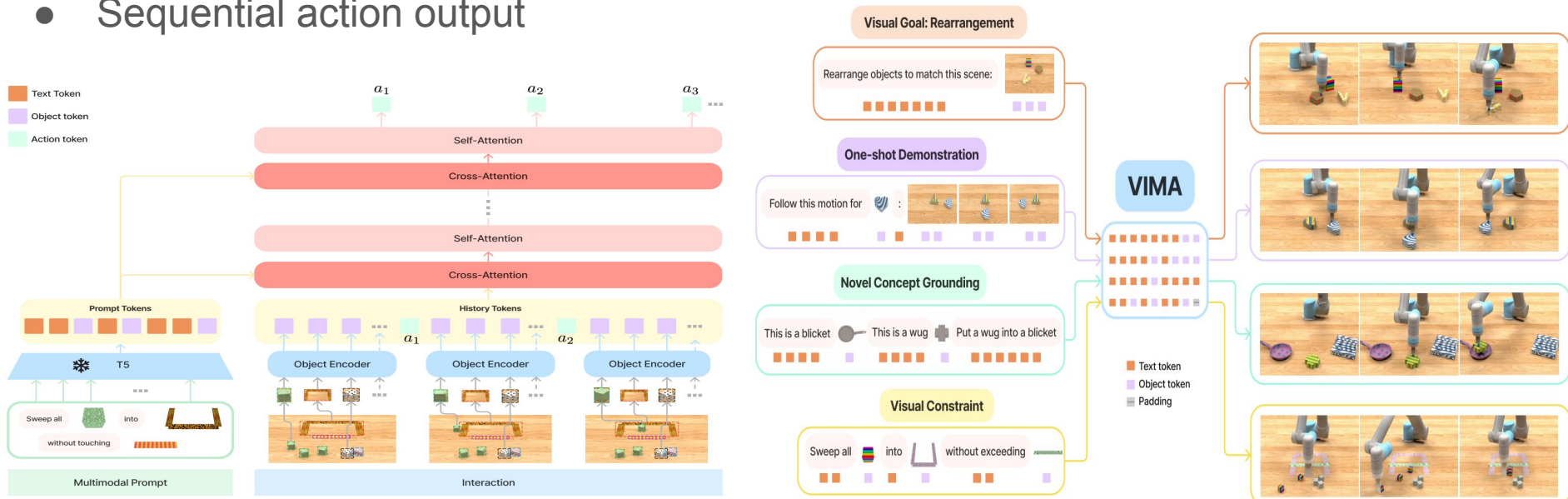
Scalability: Performance of transformers has been shown to improve smoothly as the size of the model, dataset, and compute increases. This ability can potentially be leveraged in RL to create generalist agents capable of performing various tasks in different environments and with different embodiments.

Transformers in RL

VIMA: GENERAL ROBOT MANIPULATION WITH MULTIMODAL PROMPTS

Key ideas:

- Transformer as a backbone
- Multimodal prompt
- Sequential action output

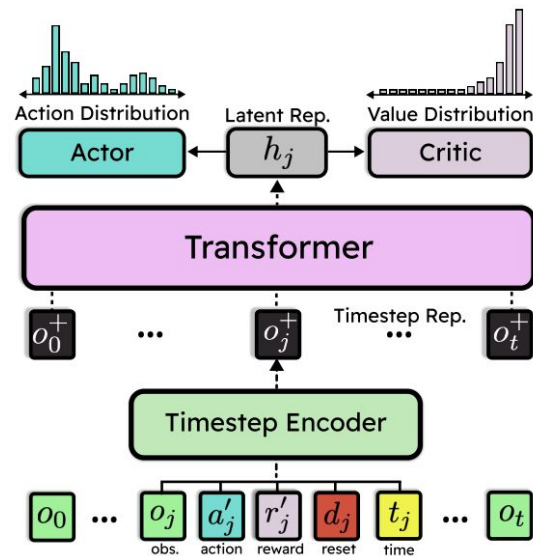
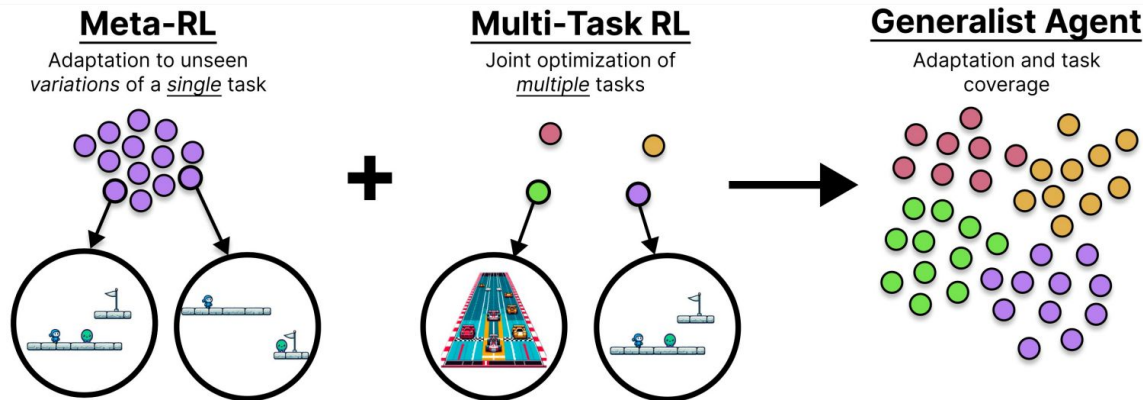


Transformers in RL

AMAGO-2: Breaking the Multi-Task Barrier in Meta-Reinforcement Learning with Transformers

Key ideas:

- Meta RL allows us to adapt to environmental shifts
- Multi-Task RL allows us to train on multiple tasks



Offline RL

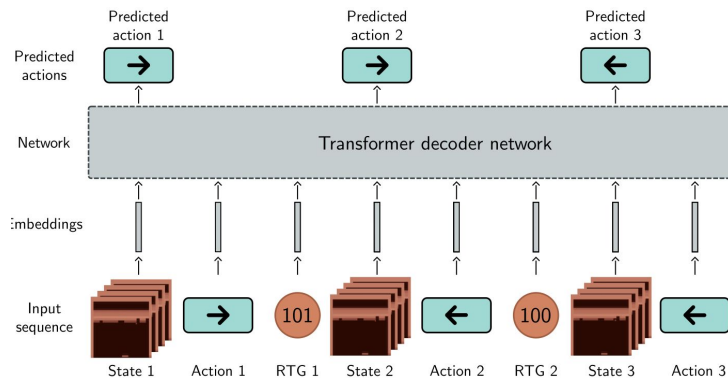
Decision Transformer: Reinforcement Learning via Sequence Modeling

Advantages:

- Safe & cheap RL
- Ability to reduce the task to sequence modeling and use already developed techniques of training and stabilization of the model

Difficulties:

- Need for a dataset and guarantees of its quality
- Small environment shifts could lead to dramatic quality loss



Hybrid of Online and Offline RL

Online Decision Transformer

Key ideas:

- 1) Stochastic policy:
$$\pi_{\theta}(a_t | \mathbf{s}_{-K,t}, \mathbf{g}_{-K,t}) = \mathcal{N}(\mu_{\theta}(\mathbf{s}_{-K,t}, \mathbf{g}_{-K,t}), \Sigma_{\theta}(\mathbf{s}_{-K,t}, \mathbf{g}_{-K,t})), \forall t,$$
- 2) Offline dataset Neg. log likelihood min-tion:
$$\begin{aligned} J(\theta) &= \frac{1}{K} \mathbb{E}_{(\mathbf{a}, \mathbf{s}, \mathbf{g}) \sim \mathcal{T}} [-\log \pi_{\theta}(\mathbf{a} | \mathbf{s}, \mathbf{g})] \\ &= \frac{1}{K} \mathbb{E}_{(\mathbf{a}, \mathbf{s}, \mathbf{g}) \sim \mathcal{T}} [-\sum_{k=1}^K \log \pi_{\theta}(a_k | \mathbf{s}_{-K,k}, \mathbf{g}_{-K,k})] \end{aligned}$$
- 3) Exploration-exploitation trade-off:
$$\begin{aligned} H_{\theta}^{\mathcal{T}}[\mathbf{a} | \mathbf{s}, \mathbf{g}] &= \frac{1}{K} \mathbb{E}_{(\mathbf{s}, \mathbf{g}) \sim \mathcal{T}} [H[\pi_{\theta}(\mathbf{a} | \mathbf{s}, \mathbf{g})]] \\ &= \frac{1}{K} \mathbb{E}_{(\mathbf{s}, \mathbf{g}) \sim \mathcal{T}} [\sum_{k=1}^K H[\pi_{\theta}(a_k | \mathbf{s}_{-K,k}, \mathbf{g}_{-K,k})]] \end{aligned}$$
- 4) Objective:
$$\min_{\theta} J(\theta) \text{ subject to } H_{\theta}^{\mathcal{T}}[\mathbf{a} | \mathbf{s}, \mathbf{g}] \geq \beta,$$

Offline pretrain + Online finetune

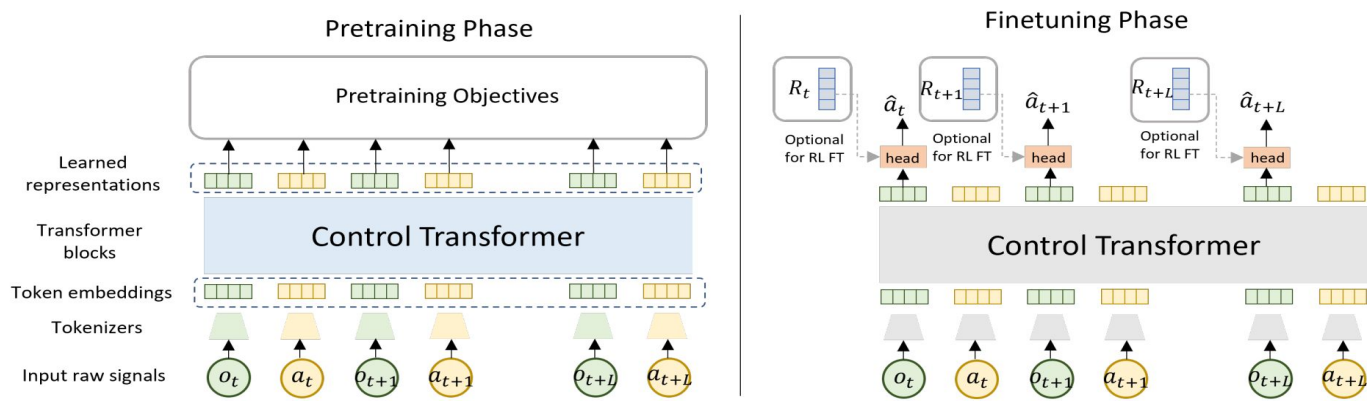
SMART: SELF-SUPERVISED MULTI-TASK PRETRAINING WITH CONTROL TRANSFORMERS

Advantages:

- Application of self-supervised and no environmental shifts
- Safer & cheaper RL

Difficulties:

- Need for a dataset and guarantees of its quality

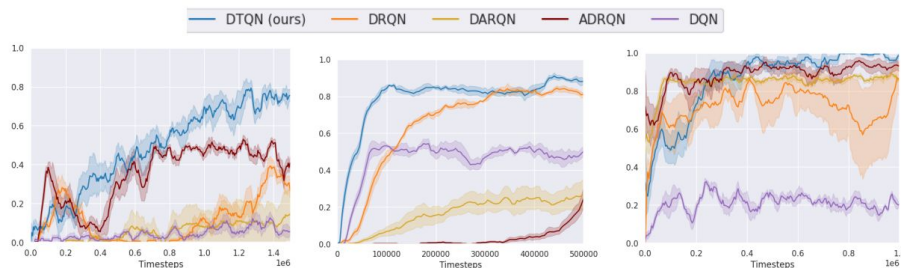


Online RL

Deep Transformer Q-Networks for Partially Observable Reinforcement Learning

Key ideas:

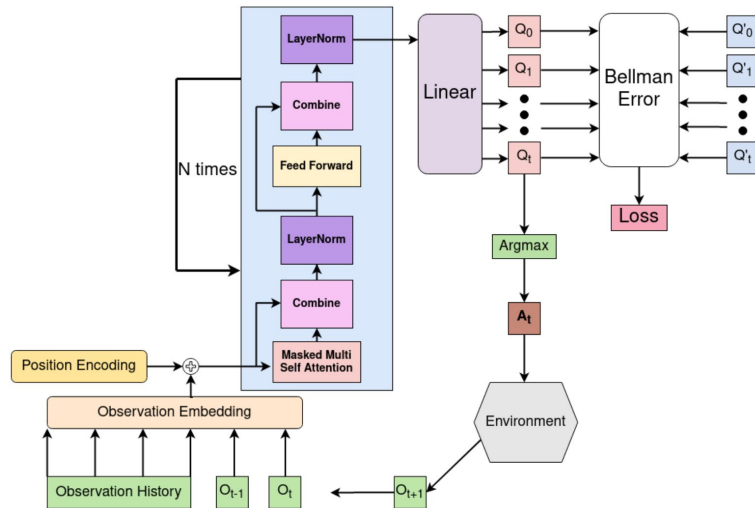
- Standard DQN pipeline
- Decoder backbone
- TD-error calculation along all actions
- Poor results (ಠ_ಠ)



(a) Gridverse memory 7x7

(b) Memory cards

(c) Hallway



Online RL

Expectations:

- Elimination of previous drawbacks
- The ability to regulate environment exploration
- The ability to fully manage the learning process

Reality:

- Usually unstable behavior during training
- Gradient vanishing
- Getting into local optima
- Low Sample-Efficiency

TOP-ERL: TRANSFORMER-BASED OFF-POLICY EPISODIC REINFORCEMENT LEARNING

Core idea in Episodic RL: $\pi(a|s) = \{a_1, a_2, \dots, a_N\}$

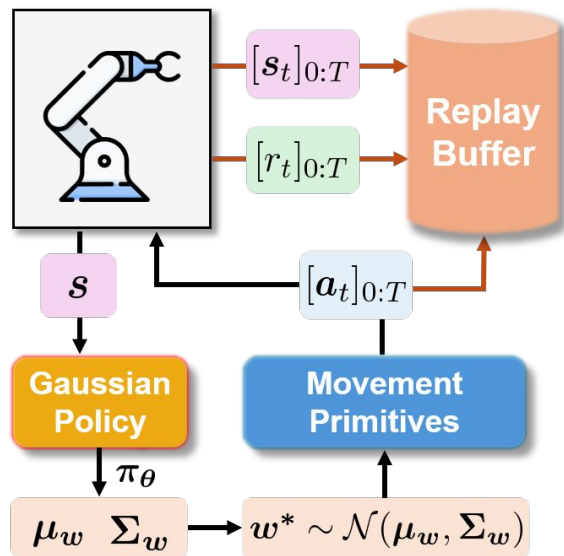
In order to generate sequence of actions let's use Movement Primitives (MPs) ([Paraschos et al., 2013](#)),

Where MP's is $F : \mathbb{R}^W \rightarrow \{a_1, a_2, \dots, a_N\}$

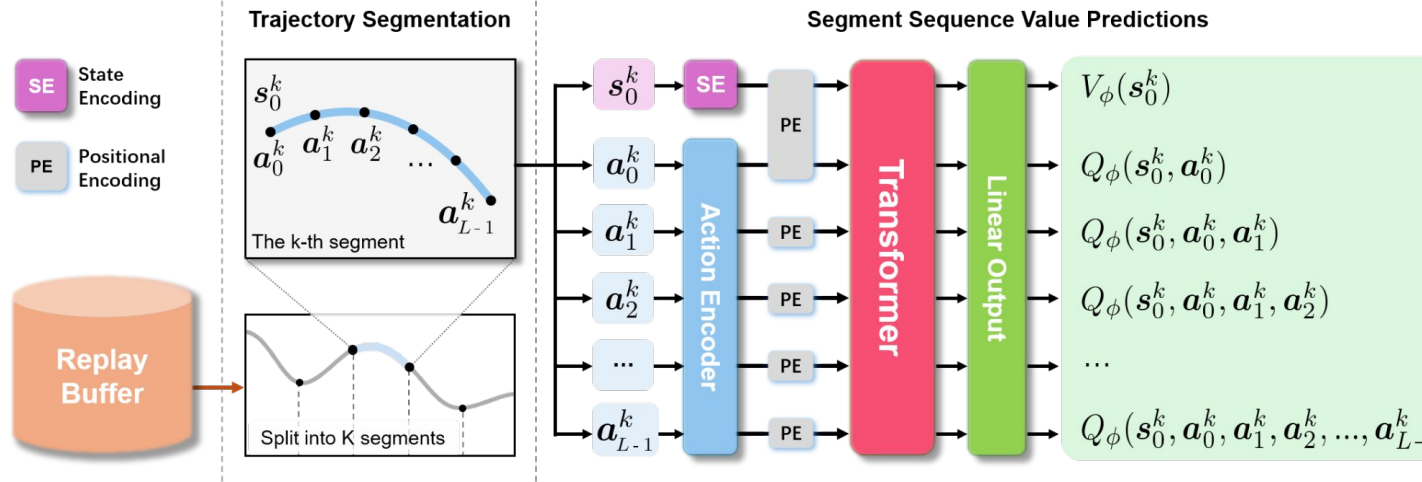
Now policy has this kind of structure:

$$\pi_{\theta}(w|s) = \mathcal{N}(w|\mu_w, \Sigma_w)$$

Policy Objective: $J(\theta) = \mathbb{E}_{s \sim B} \mathbb{E}_{\tilde{w} \sim \pi_{\theta}(\cdot|s)} \left[\frac{1}{KL} \sum_{k=1}^K \sum_{N=0}^{L-1} Q_{\phi}(s_0^k, [\tilde{a}_t^k]_{t=0:N}) \right]$



How does the Critic work ?



$$\text{Critic loss: } \mathcal{L}(\phi) = \frac{1}{L} \sum_{N=1}^{L-1} \left[\underbrace{Q_\phi(s_0^k, a_0^k, \dots, a_{N-1}^k)}_{\text{Predicted value of N actions}} - \underbrace{G^{(N)}(s_0^k, a_0^k, \dots, a_{N-1}^k)}_{\text{Target using N-step return}} \right]^2$$

$$+ \left[\underbrace{V_\phi(s_0^k)}_{\text{Predicted state value}} - \underbrace{\mathbb{E}_{\tilde{w} \sim \pi_\theta(\cdot|s)} [Q_{\phi_{\text{tar}}}(s_0^k, \tilde{a}_0^k, \dots, \tilde{a}_{L-1}^k)]}_{\text{Target of new actions using } \tilde{w}} \right]^2$$

$$\text{N-step return: } G^{(N)}(s_0^k, a_0^k, \dots, a_{N-1}^k) = \underbrace{\sum_{i=0}^{N-1} \gamma^i r_i}_{\text{N-step rewards}} + \underbrace{\gamma^N V_{\phi_{\text{tar}}}(s_N)}_{\text{Future return after N-step}}$$

TOP-ERL Algorithm

Algorithm 1 TOP-ERL

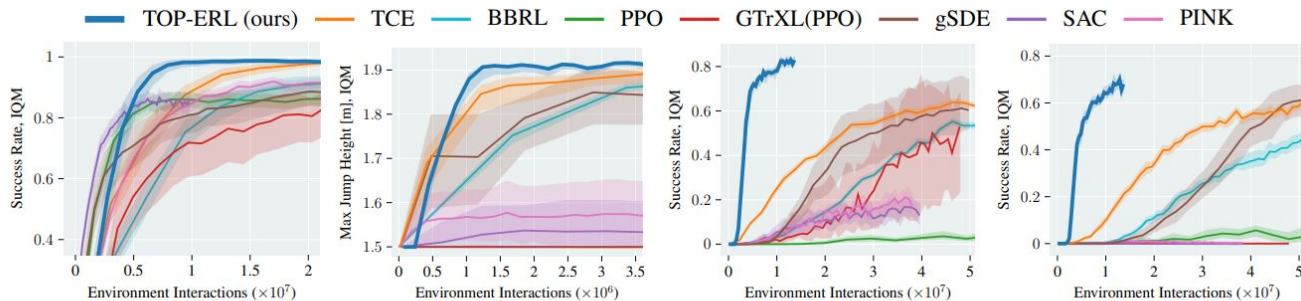
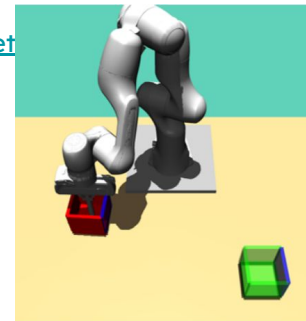
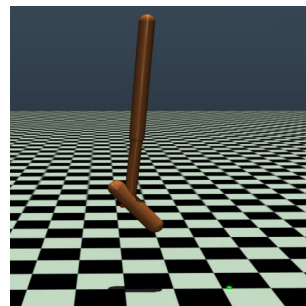
- 1: Initialize critic ϕ ; target critic $\phi_{\text{tar}} \leftarrow \phi$
 - 2: Initialize policy θ and replay buffer $\mathcal{B}$
 - 3: **repeat**
 - 4: Reset environment and get initial task state $\mathbf{s}$
 - 5: Predict the policy mean $\boldsymbol{\mu}_w$ and covariance $\boldsymbol{\Sigma}_w$
 - 6: Sample w^* and generate action trajectory $[\mathbf{a}]_{0:T}$
 - 7: Execute the action trajectory till task ends.
 - 8: Store the visited states $[\mathbf{s}]_{0:T}$, rewards $[r]_{0:T}$, and the action $[\mathbf{a}]_{0:T}$ trajectories in replay buffer $\mathcal{B}$
 - 9: **for** each update step **do**
 - 10: From $\mathcal{B}$, sample a batch of $\mathbf{s}, \mathbf{a}, r$ trajectories.
 - 11: Split them into K segments, each L time steps.
 - 12: Compute N-step return targets as in Eq.(8)
 - 13: Update transformer critic, using Eq.(7)
 - 14: Update policy, using Eq.(9)
 - 15: **end for**
 - 16: Update target critic $\phi_{\text{tar}} \leftarrow (1 - \rho) \phi_{\text{tar}} + \rho \phi$
 - 17: **until** converged
-

TOP-ERL Results

- Can TOP-ERL improve sample efficiency in classical ERL tasks characterized by challenging exploration problems?
- How does TOP-ERL perform in large-scale, general manipulation benchmarks?

Environments:

- Meta-World MT50 ([Yu et al., 2020](#))
- Hopper Jump: a sparse-reward environment where the objective is to maximize the jump height within an episode ([Li et al., 2024](#))
- Box Pushing with sparse rewards
- Box Pushing with dense rewards



(a) Metaworld Agg.
of 50 tasks

(b) Hopper Jump
Max Height

(c) Box Pushing
Dense Reward

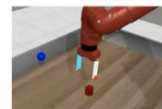
(d) Box Pushing
Sparse Reward



basketball



button press



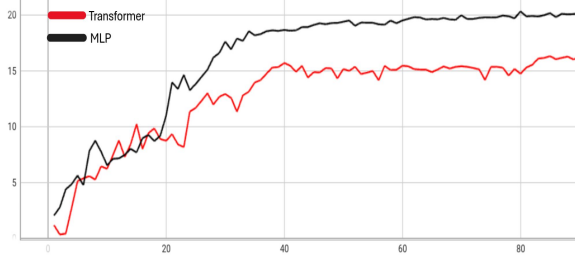
pick place



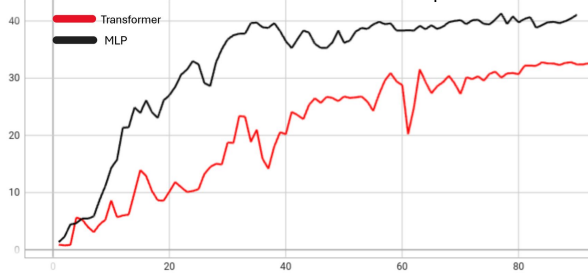
push

My current results in Online Transformer RL (TD3)

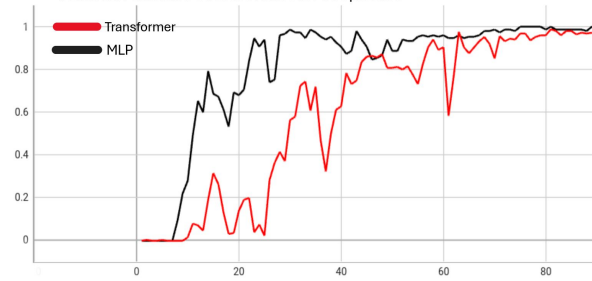
ManiSkill PickCube Evaluation Reward over 10 episodes



ManiSkill PushCube Evaluation Reward over 10 episodes



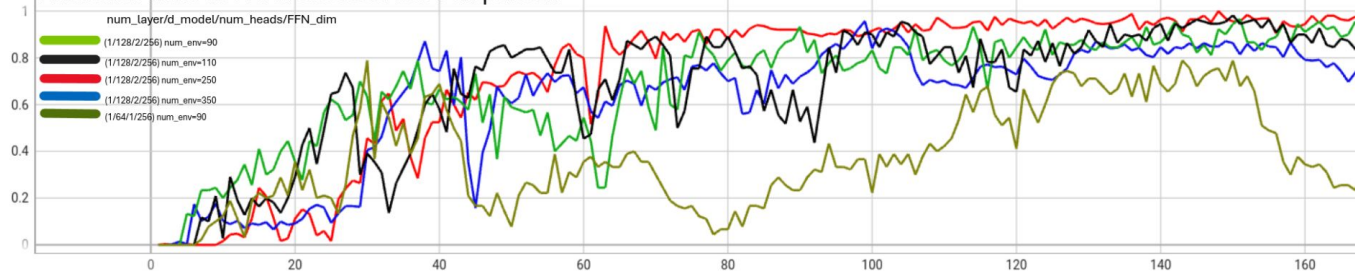
ManiSkill PushCube Success Rate over 10 episodes



ManiSkill PushCube Eval Reward comparison

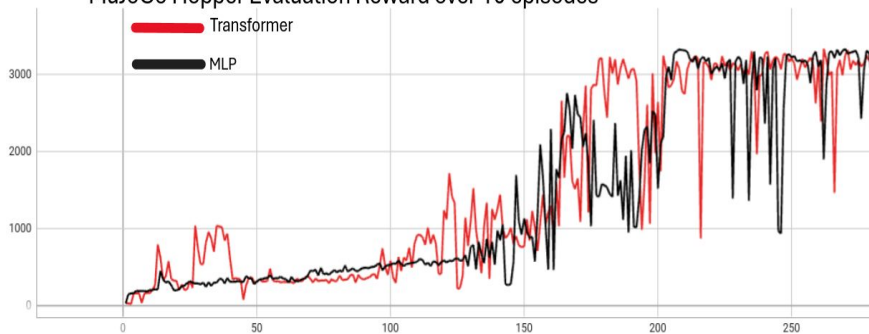


ManiSkill PushCube Eval SuccessRate comparison

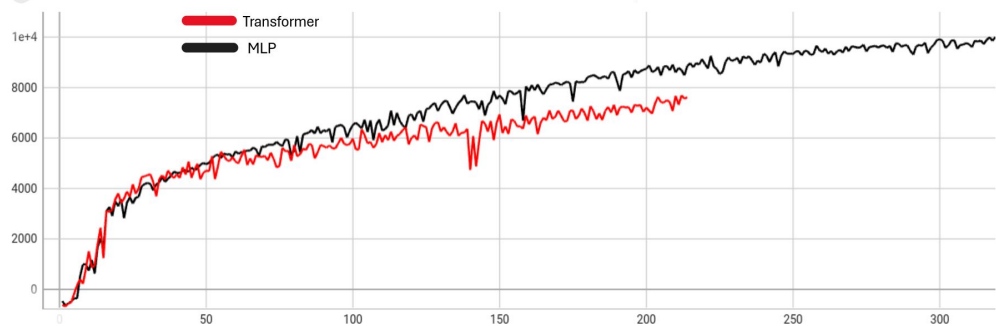


MuJoCo Hopper and Half-Cheetah successful learning

MuJoCo Hopper Evaluation Reward over 10 episodes



MuJoCo Half-Cheetah Evaluation Reward over 10 episodes



14

REINFORCEMENT LEARNING & TRANSFORMER MODELS: TRANSFORMER-BASED ONLINE RL

ДАНИИЛ
ЗЕЛЕЗЕЦКИЙ



НИКИТА
КАЧАЕВ



12 ДЕКАБРЯ
17:00



Center for
Cognitive
Modeling



Cognitive
Dynamic
Systems



ITL

Intelligent
Transport
Lab



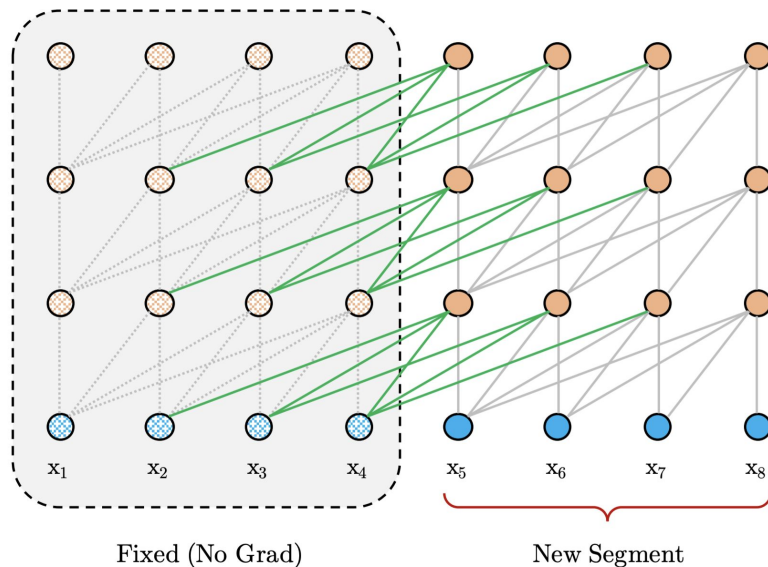
ИНСТИТУТ
ИСКУССТВЕННОГО
ИНТЕЛЛЕКТА
МФТИ

ФПМИ



МФТИ

Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context [1]



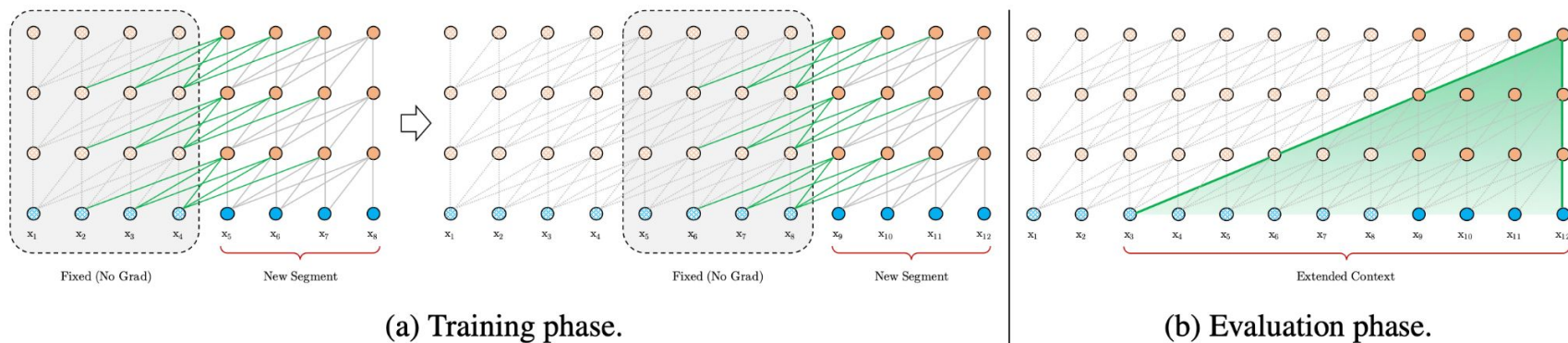
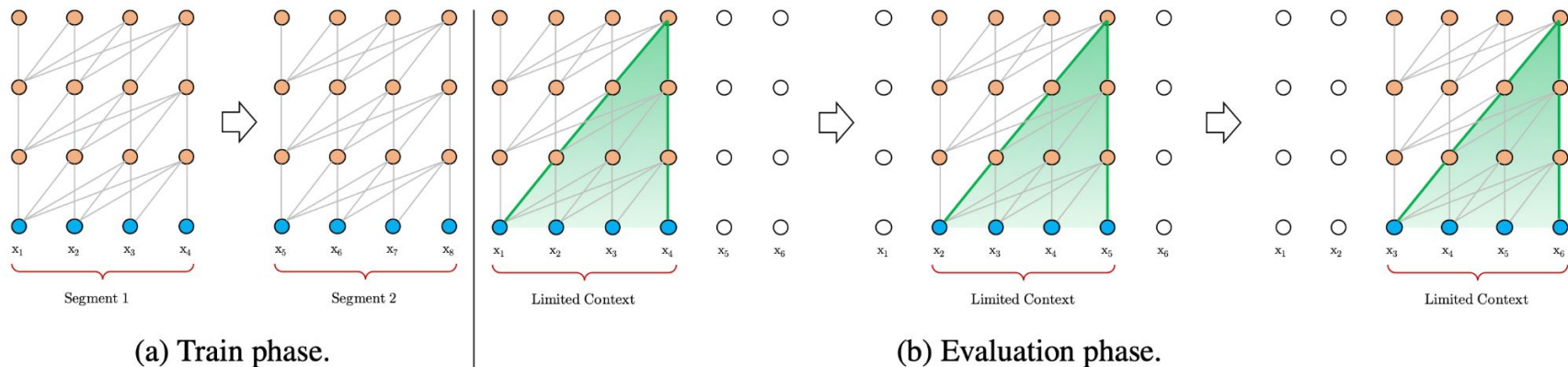
$$\tilde{\mathbf{h}}_{\tau+1}^{n-1} = [\text{SG}(\mathbf{h}_{\tau}^{n-1}) \circ \mathbf{h}_{\tau+1}^{n-1}] ,$$

$$\mathbf{q}_{\tau+1}^n, \mathbf{k}_{\tau+1}^n, \mathbf{v}_{\tau+1}^n = \mathbf{h}_{\tau+1}^{n-1} \mathbf{W}_q^\top, \tilde{\mathbf{h}}_{\tau+1}^{n-1} \mathbf{W}_k^\top, \tilde{\mathbf{h}}_{\tau+1}^{n-1} \mathbf{W}_v^\top ,$$

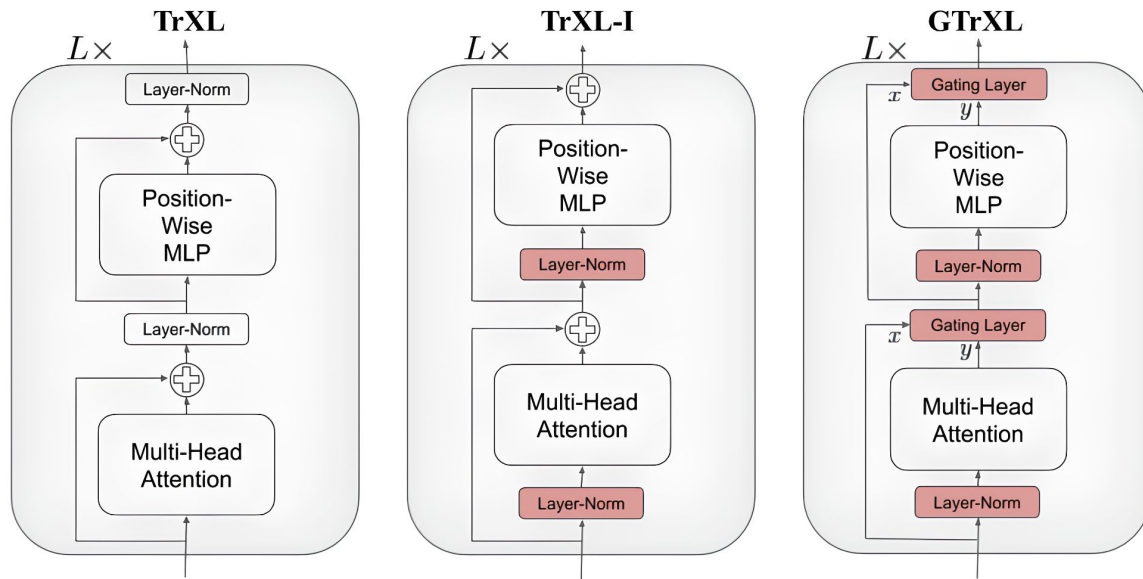
$$\mathbf{h}_{\tau+1}^n = \text{Transformer-Layer}(\mathbf{q}_{\tau+1}^n, \mathbf{k}_{\tau+1}^n, \mathbf{v}_{\tau+1}^n) .$$

Consequently, the largest possible dependency length grows linearly w.r.t. the number of layers as well as the segment length, i.e., $O(N \times L)$

Transformer-XL vs Vanilla Transformer train and evaluation:



GTrXL: Stabilizing transformers for Reinforcement learning [2]



- Identity Map Reordering (pre-layer norm)
- Gating mechanism
- Gated Identity Initialization:

Key modifications:

Identity Map Reordering:

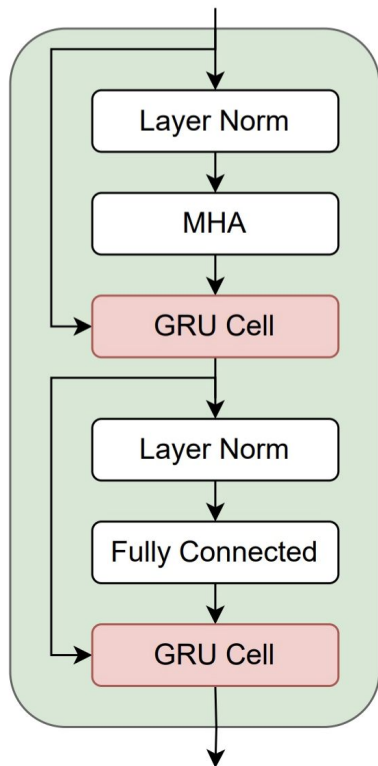
One hypothesis as to why the Identity Map Reordering improves results is as follows: assuming that the submodules at initialization produce values that are in expectation near zero, the state encoding is passed un-transformed to the policy and value heads, enabling the agent to learn a Markovian policy at the start of training (i.e., the network is initialized such that $\pi(\cdot|s_t, \dots, s_1) \approx \pi(\cdot|s_t)$ and $V \pi(s_t|s_{t-1}, \dots, s_1) \approx V \pi(s_t|s_{t-1})$). In many environments, reactive behaviours need to be learned before memory-based ones can be effectively utilized, i.e., an agent needs to learn how to walk before it can learn how to remember where it has walked.

Gating mechanism:

Gated Identity Initialization:

We have claimed that the Identity Map Reordering aids policy optimization because it initializes the agent close to a Markovian policy / value function. If this is indeed the cause of improved stability, we can explicitly initialize the various gating mechanisms to be close to the identity map. This is the purpose of the bias $b(l)_g$ in the applicable gating layers. We later demonstrate in an ablation that initially setting $b(l)_g > 0$ can greatly improve learning speed.

GTrXL layer:



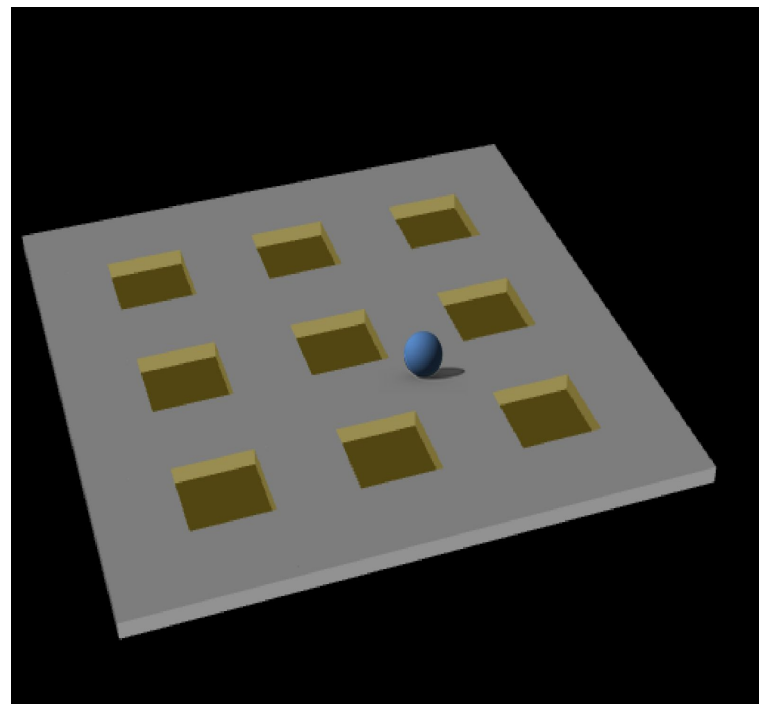
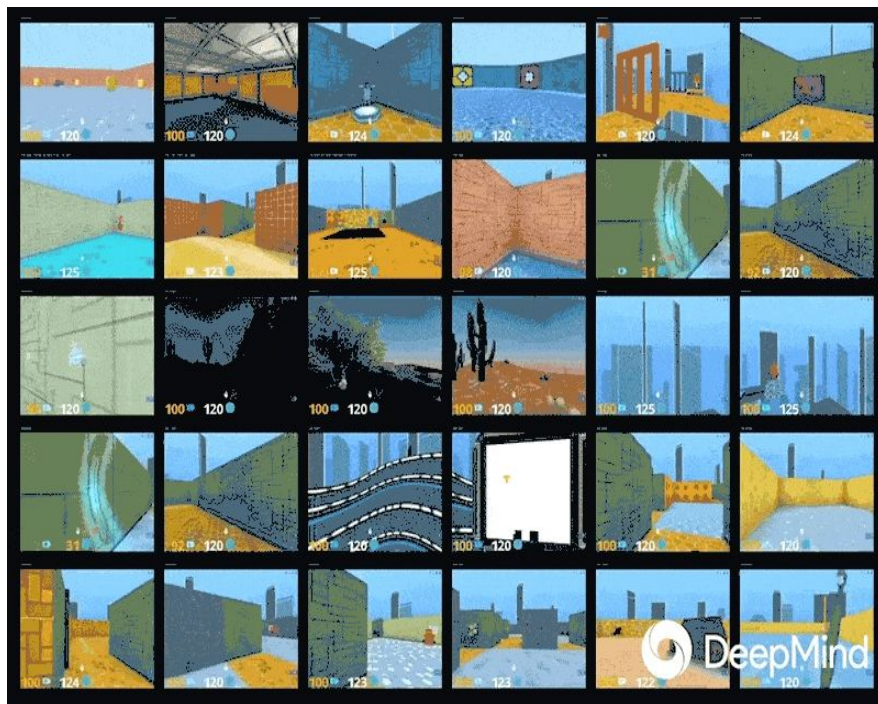
$$\bar{Y}^{(l)} = \text{RelativeMultiHeadAttention}(\text{LayerNorm}([\text{StopGrad}(M^{(l-1)}), E^{(l-1)}]))$$

$$Y^{(l)} = g_{\text{MHA}}^{(l)}(E^{(l-1)}, \text{ReLU}(\bar{Y}^{(l)}))$$

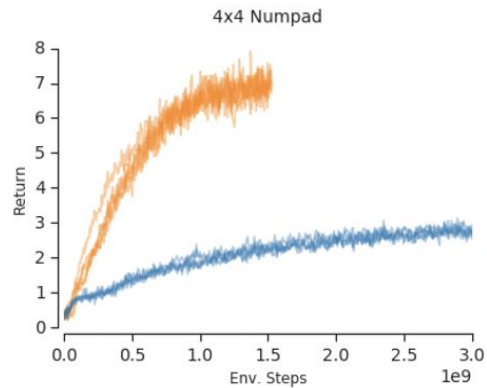
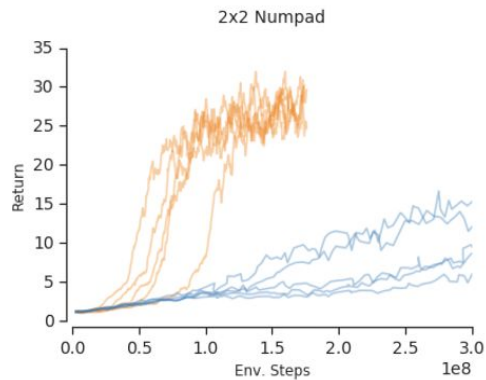
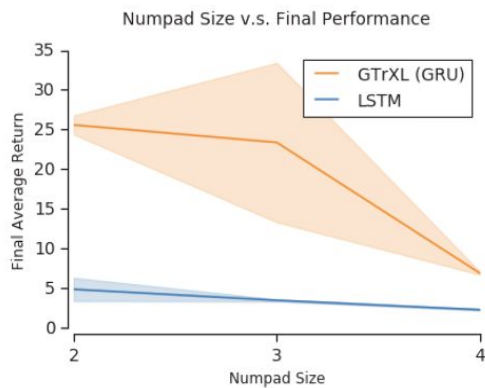
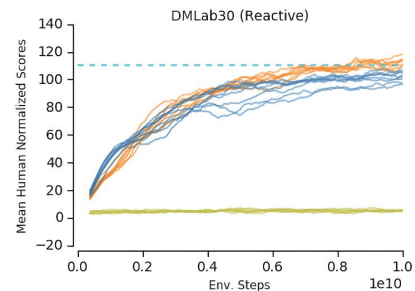
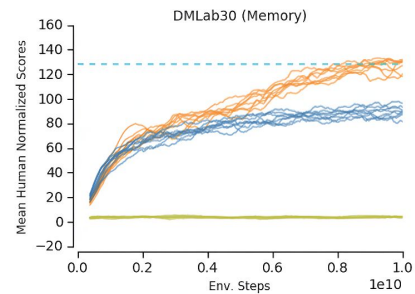
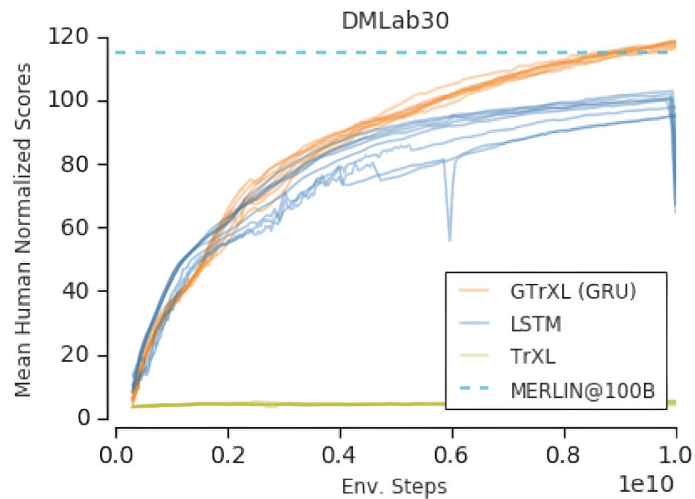
$$\bar{E}^{(l)} = f^{(l)}(\text{LayerNorm}(Y^{(l)}))$$

$$E^{(l)} = g_{\text{MLP}}^{(l)}(Y^{(l)}, \text{ReLU}(\bar{E}^{(l)}))$$

Environments:



Results:



Ablation on gating mechanism: gating types

Input: The gated input connection has a sigmoid modulation on the input stream:

$$g^{(l)}(x, y) = \sigma(W_g^{(l)}x) \odot x + y$$

Output: The gated output connection has a sigmoid modulation on the output stream:

$$g^{(l)}(x, y) = x + \sigma(W_g^{(l)}x - b_g^{(l)}) \odot y$$

Highway: The highway connection modulates both streams with a sigmoid:

$$g^{(l)}(x, y) = \sigma(W_g^{(l)}x + b_g^{(l)}) \odot x + (1 - \sigma(W_g^{(l)}x + b_g^{(l)})) \odot y$$

Sigmoid-Tanh: similar to the **Output** gate but with an additional tanh activation on the output stream

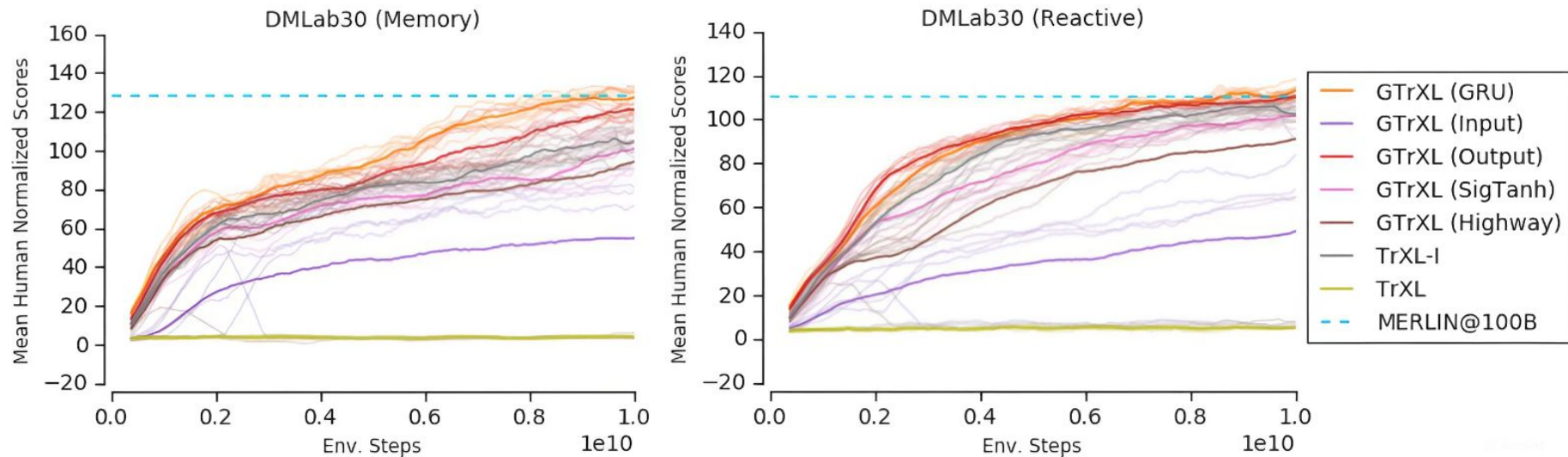
$$g^{(l)}(x, y) = x + \sigma(W_g^{(l)}y - b) \odot \tanh(U_g^{(l)}y)$$

Gated-Recurrent-Unit-type gating:

$$r = \sigma(W_r^{(l)}y + U_r^{(l)}x) \quad z = \sigma(W_z^{(l)}y + U_z^{(l)}x - b_g^{(l)}) \quad \hat{h} = \tanh(W_g^{(l)}y + U_g^{(l)}(r \odot x))$$

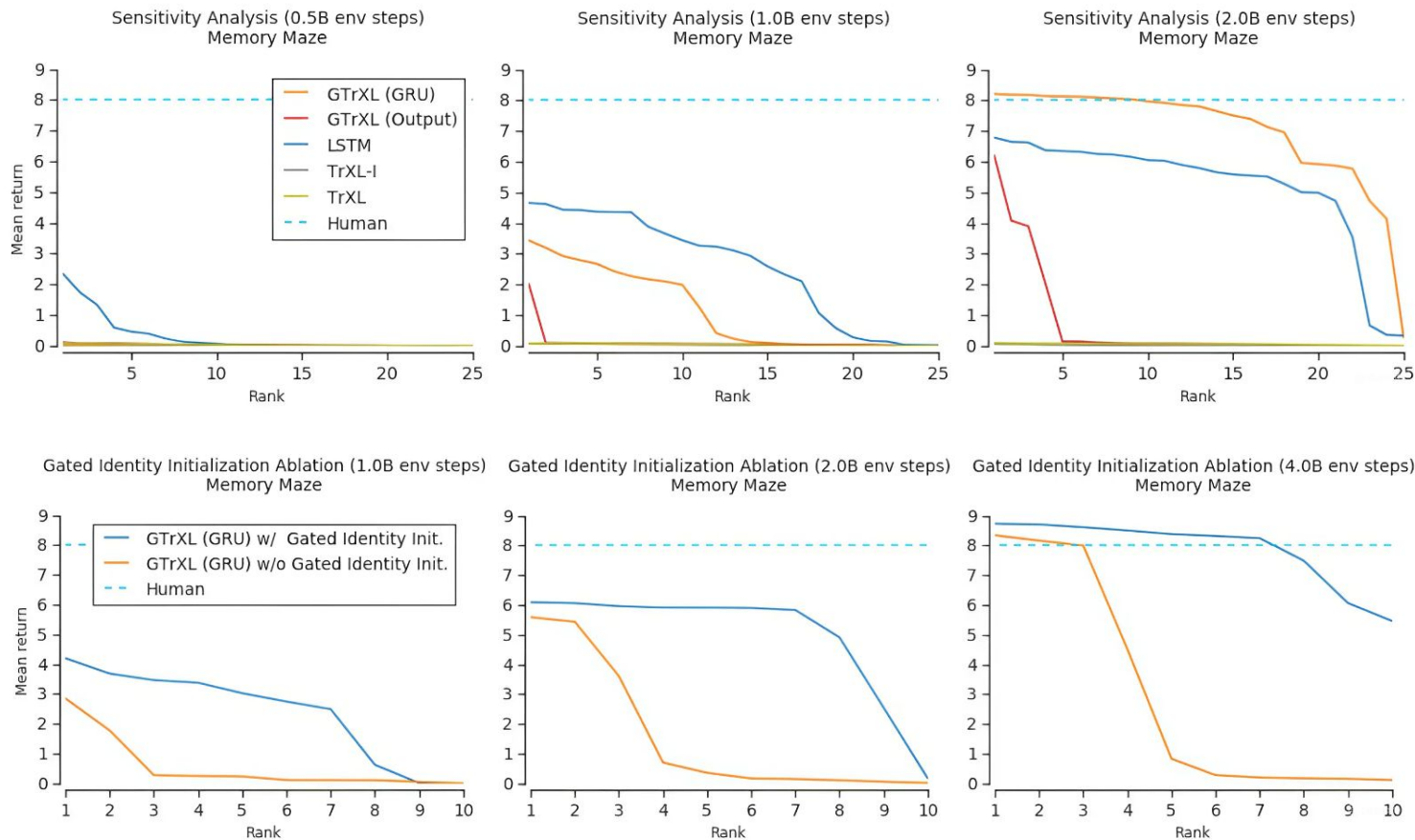
$$g^{(l)}(x, y) = (1 - z) \odot x + z \odot \hat{h}$$

Ablation on gating mechanism: results



Model	Mean Human Norm.
LSTM	99.3 ± 1.0
TrXL	5.0 ± 0.2
TrXL-I	107.0 ± 1.2
MERLIN@100B	115.2
GTrXL (GRU)	117.6 ± 0.3
GTrXL (Input)	51.2 ± 13.2
GTrXL (Output)	112.8 ± 0.8
GTrXL (Highway)	90.9 ± 12.9
GTrXL (SigTanh)	101.0 ± 1.3

Hyperparameters sensitivity analysis:



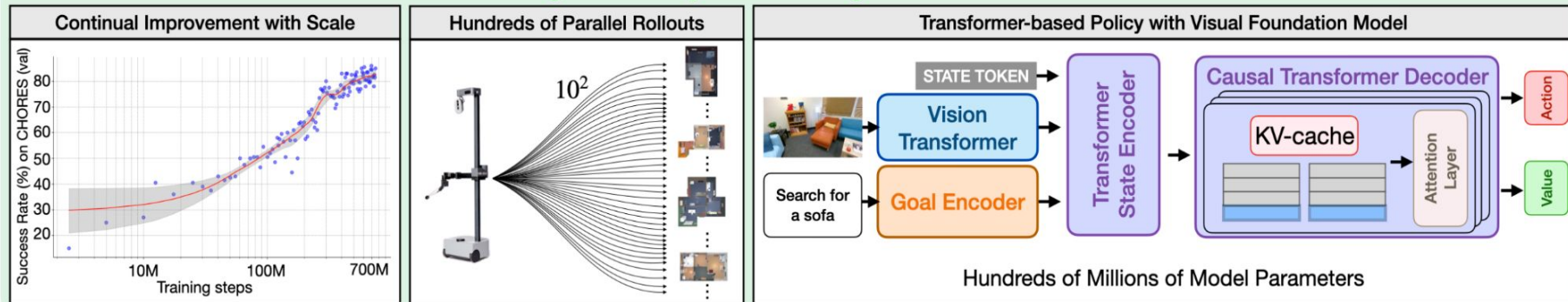
Stabilizing transformers for Reinforcement learning:

Model	# Layers	Head Dim.	# Heads	Hidden Dim.	Memory Size	Runs Completed
LSTM	3	-	-	256	-	8
TrXL	12	64	8	256	512	6
TrXL-I	12	64	8	256	512	6
GTrXL (GRU)	12	64	8	256	512	8

Model	Mean Human Norm. Score	# Param.
LSTM	99.3 ± 1.0	9.25M
TrXL	5.0 ± 0.2	28.6M
TrXL-I	107.0 ± 1.2	28.6M
Thin GTrXL (GRU)	111.5 ± 0.6	22.4M
GTrXL (GRU)	117.6 ± 0.3	66.4M
GTrXL (Input)	51.2 ± 13.2	34.9M
GTrXL (Output)	112.8 ± 0.8	34.9M
GTrXL (Highway)	90.9 ± 12.9	34.9M
GTrXL (SigTanh)	101.0 ± 1.3	41.2M

PoliFormer: Scaling On-Policy RL with Transformers Results in Masterful Navigators [3]

Scaling On-Policy RL Training with Transformers



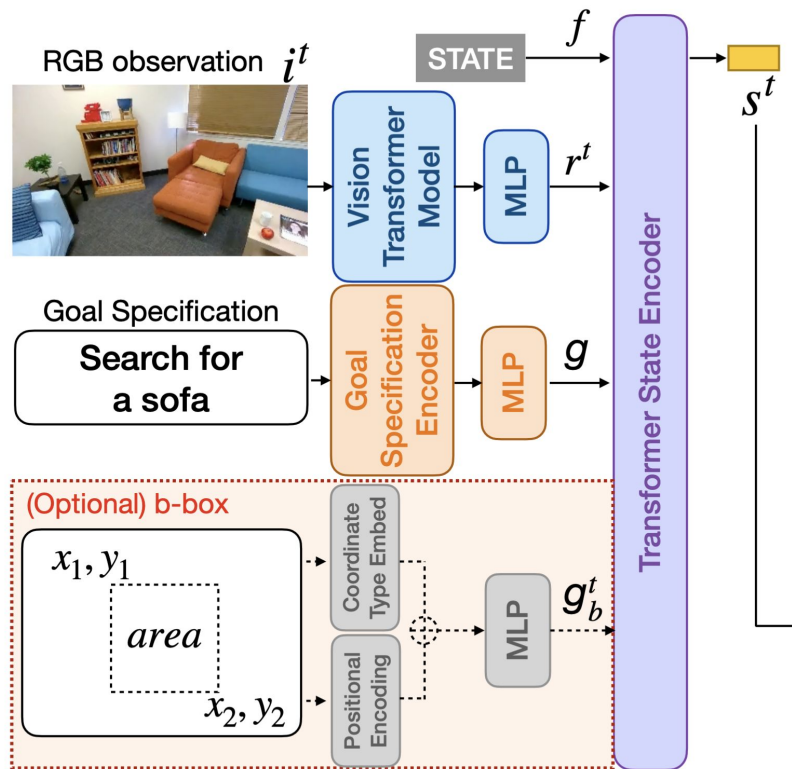
Success recipe - scale learning with transformers across three domains:

1. **Scale in Architecture:** higher network capacity, via PoliFormer's deep, high-capacity, model architecture
2. **Scale in Rollouts:** parallelism and batch size via highly-parallelized training methodology that leverages large batch sizes for efficiency
3. **Scale in Diverse Environment Interactions:** optimization of the simulation environment to support high-speed training

[3] PoliFormer: Scaling On-Policy RL with Transformers Results in Masterful Navigators

[arXiv:2406.20083](https://arxiv.org/abs/2406.20083)

Encoder:



Vision Transformer Model:

- **DINOv2** as our **visual foundation backbone**
- For effective **sim-to-real** transfer, it is important to **keep** the vision transformer model **frozen** when training in simulation.

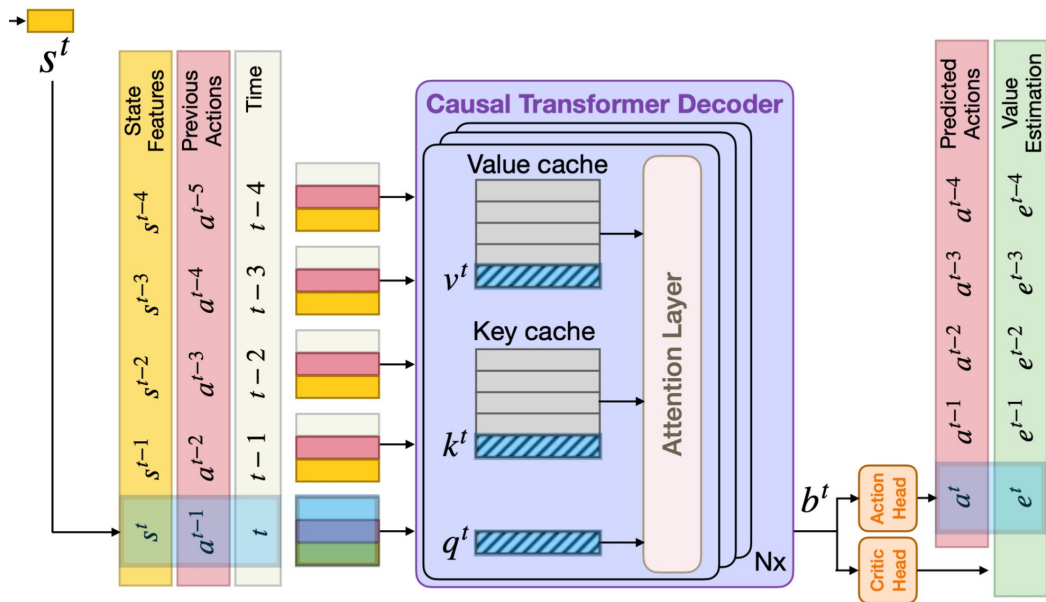
Goal Encoder:

- **EmbCLIP** for benchmarks with **LoCoBot** and **FLAN-T5** small for **Stretch RE-1**

Transformer State Encoder:

- This module **summarizes the state** at each timestep as a vector. The input to this encoder includes the visual representation, the goal feature and embedding of a STATE token

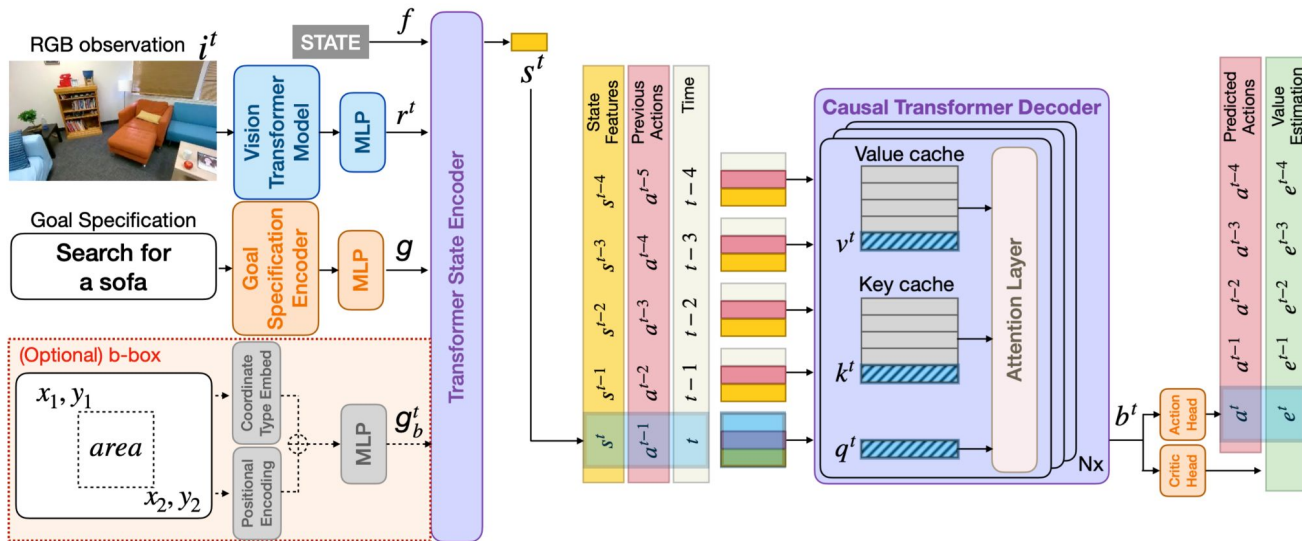
Decoder:



Causal Transformer Decoder:

- Constructs its state belief using the sequence of state features.
- Replace the **standard transformer decoder** block with the implementation used in the **Llama 2 LLM**. This change has **dramatic implications** for training and inference speed

Architecture scaling:



Model			PROCTHOR-10k (val)	ARCHITECTHOR
Vision Backbone	Encoder	Decoder	Success	
CLIP (ResNet50)	1x CNN	1x GRU	67.7	55.8
DINOv2 (ViTs)	1x CNN	1x GRU	73.1	60.8
DINOv2 (ViTs)	3x Tx	3x GRU	73.6	59.8
DINOv2 (ViTs)	3x Tx	1x Tx	77.2	59.1
DINOv2 (ViTs)	3x Tx	3x Tx	80.4	63.9
DINOv2 (ViTb)	3x Tx	3x Tx	82.4	68.3

(a) Ablations on design choices for scaling model capacity

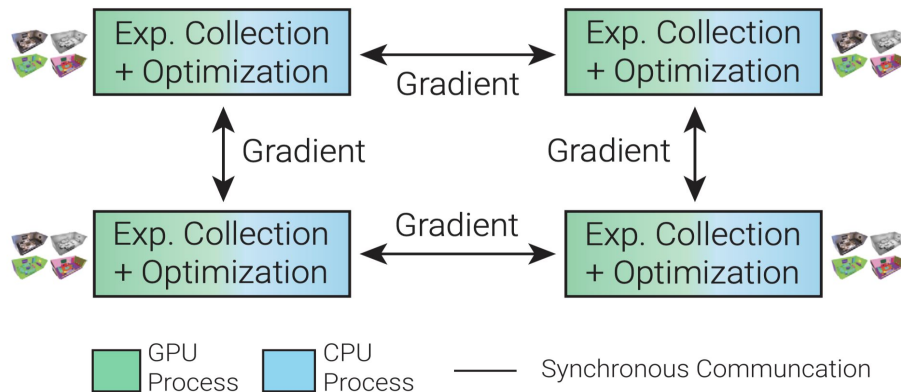
RL training scaling

- Multi-node training (increasing the number of parallel rollouts by a factor of 4 compared to previous approaches)
 - **DD-PPO** across **4 nodes**
 - 4.5 days with 4 nodes, while this would have required 15.3 days with a single node
 - total of **32 A6000 GPUs** and **512 CPU cores**
- This scaling accelerates the training speed by approximately 3.4×, a near-linear gain compared to single-node training
- Batch size
 - Is equal to the number of parallel rollouts (R) multiplied by the length of these rollouts (T) and thus, by increasing R as above, we multiplicatively increase the total batch size.
 - **Instead of annealing the learning rate** during training, **increase the batch size [4]** by changing the rollout length from T=32, to T=64, and finally to T=128 (resulting in a final batch size of 49,152 frames for LoCoBot agents). Make these increases every 10M steps until we reach T=128

[5] Don't Decay the Learning Rate, Increase the Batch Size

[arXiv:1711.00489](https://arxiv.org/abs/1711.00489)

RL training scaling: DD-PPO (Decentralized Distributed PPO) [5]



We adapt this to on-policy RL as follows: At step k , a worker n has a copy of the parameters θ_n^k ; it gathers experience (rollout) using $\pi_{\theta_n^k}$, calculates the parameter-gradients ∇_{θ} via any policy-gradient method (*e.g.* PPO), synchronizes these gradients with other workers, and updates the model:

$$\theta_n^{k+1} = \text{ParamUpdate}\left(\theta_n^k, \text{AllReduce}\left(\nabla_{\theta}\mathcal{J}^{PPO}(\theta_1^k), \dots, \nabla_{\theta}\mathcal{J}^{PPO}(\theta_N^k)\right)\right). \quad (4)$$

Scaling Environment interactions



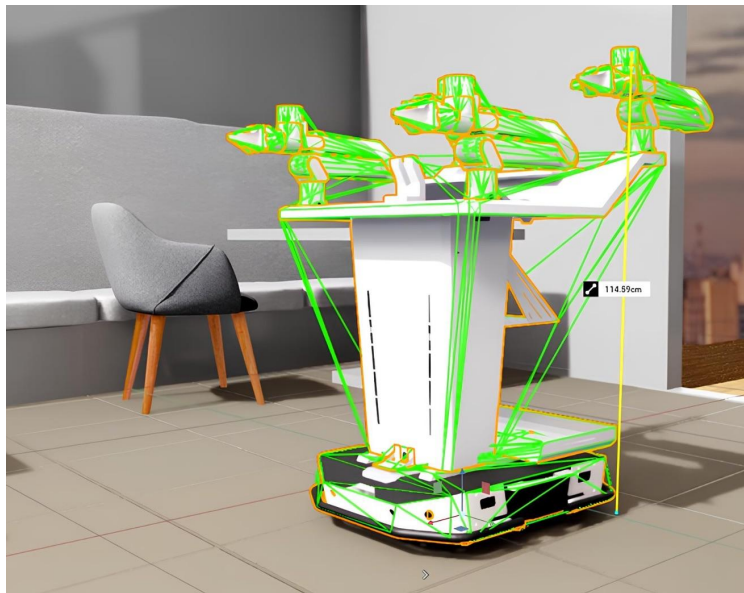
PoliFormer was trained with RL at scale in 150k **procedurally generated** PROCTOR houses using **optimized Objaverse assets** results in **steady validation set gains**

Scaling Environment interactions



AI2-THOR **scene loading and resetting** was a **significant bottleneck** when using Objaverse assets. We streamline AI2-THOR's Objaverse 3D asset loading pipeline by saving objects in the Msgpack format (natively usable with Unity's C# backend) as opposed to Python-only Pickle files. This change **dramatically decreases the loading time** of a new scene from **~25s to ~8s**.

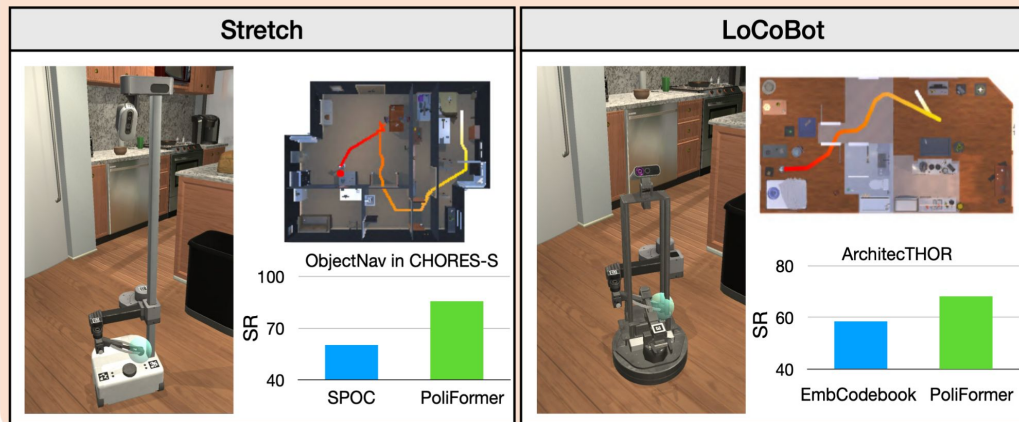
Scaling Environment interactions



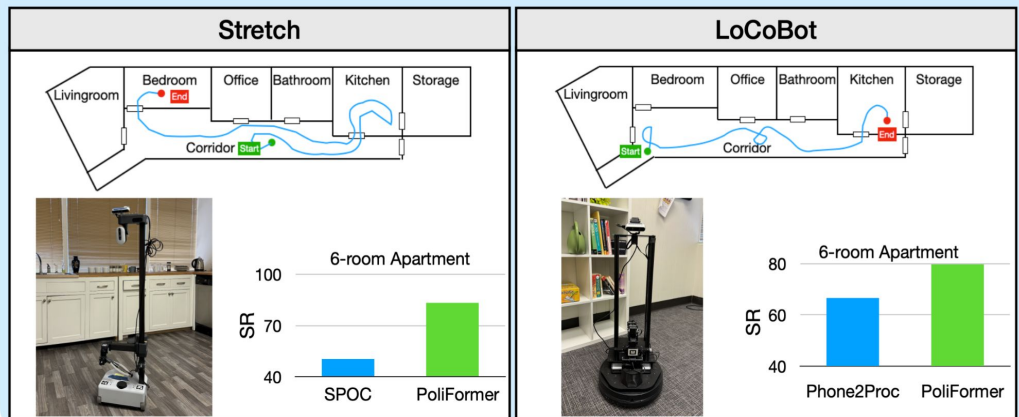
- Physics simulation used by the Stretch RE-1 agent in AI2-THOR to be **too slow to efficiently train at scale**
- Discretely approximate the agent's continuous movement via a **teleportation-with-collision-checks** approach
- Physics cast using a **capsule collider** representing the agent along the desired movement direction
- These approximations **increase simulation speed by ~40%**
- For LoCoBot, the **action space was discretized** into 6 actions, including {MoveAhead, RotateRight, RotateLeft, LookUp, LookDown, Done}

Results:

Simulation



Real



Results:

Inputs	Model	Loss	CHORES- $\mathbb{S}$ ObjectNav
			Success (SEL)
RGB+text	SPOC [6]	IL	57.0 (46.2)
	SPOC*	IL	60.0 (30.5)
	EmbSigLIP [6]	RL	36.5 (24.5)
	POLIFORMER	RL	85.5 (61.2)
RGB +text+b-box	SPOC	IL	85.0 (61.4)
	POLIFORMER	RL	95.5 (71.4)
RGB+b-box	POLIFORMER	RL	92.0 (73.9)

(a) Stretch RE-1 on CHORES- $\mathbb{S}$

Inputs	Model	PROCThor-10k	ARCHITECTHOR	AI2-iTHOR
		Success (SPL)		
RGB+text	PROCThor [11] ³	67.7 (49.0)	55.8 (38.3)	70.0 (57.1)
	SGC [63]	70.8 (48.6)	53.8 (34.8)	71.4 (59.3)
	EmbCodebook [86]	73.7 (48.4)	58.3 (35.6)	78.4 (23.7)
	POLIFORMER	82.4 (58.5)	68.3 (45.1)	85.3 (72.7)
RGB +text+b-box	POLIFORMER	90.4 (66.6)	81.9 (55.6)	94.9 (83.5)
RGB+b-box	POLIFORMER	87.4 (56.2)	85.7 (47.6)	92.1 (78.6)

(b) LoCoBot on ProcThor-10k (val), ArchitectHOR and AI2-iTHOR (test)

Model	Stretch RE-1	LoCoBot
ProcThor [11]	-	26.7
Phone2Proc [17]	-	66.7
SPOC [6]	50.0	-
POLIFORMER (ours)	83.3	80.0
SPOC+Detic [6]	83.3	-
POLIFORMER +Detic (ours)	88.9	-

(b) Real-world results - Success

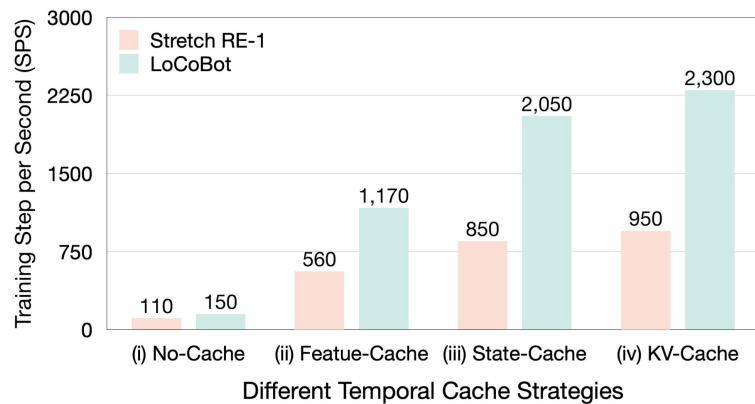
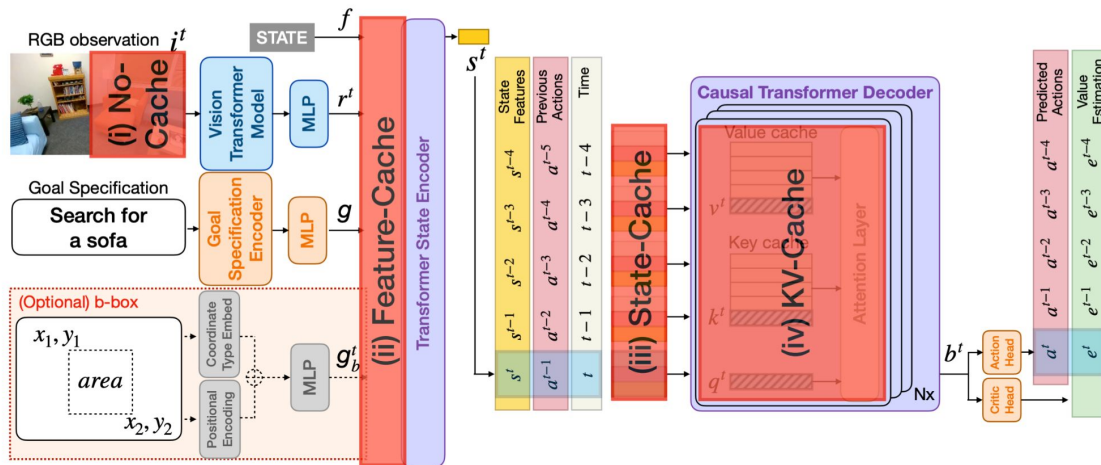
As seen from experiments, the performance of SPOC does not seem to improve significantly when further scaling up data and model depth

Hyperparameters for training and model architecture:

Training and Model Details

Parameter	Value
Allowed Steps	600 (Stretch RE-1), 500 (LoCoBot)
Total Rollouts	192 (Stretch RE-1), 384 (LoCoBot)
Learning Rate	0.002
Mini Batch per Update	1
Update Repeats	4
Max Gradient Norm	0.5
Discount Value Factor γ	0.99
GAE λ	0.95
PPO Surrogate Objective Clipping	0.1
Value Loss Weight	0.5
Entropy Loss Weight	0.01
Training Stages	3
Steps for PPO Update Stage 1	32
Steps for PPO Update Stage 2	64
Steps for PPO Update Stage 3	128
Transformer State Encoder Layers	3
Transformer State Encoder Hidden Dims	512
Transformer State Encoder Heads	8
Causal Transformer Decoder Layers	3
Causal Transformer Decoder Hidden Dims	512
Causal Transformer Decoder Heads	8

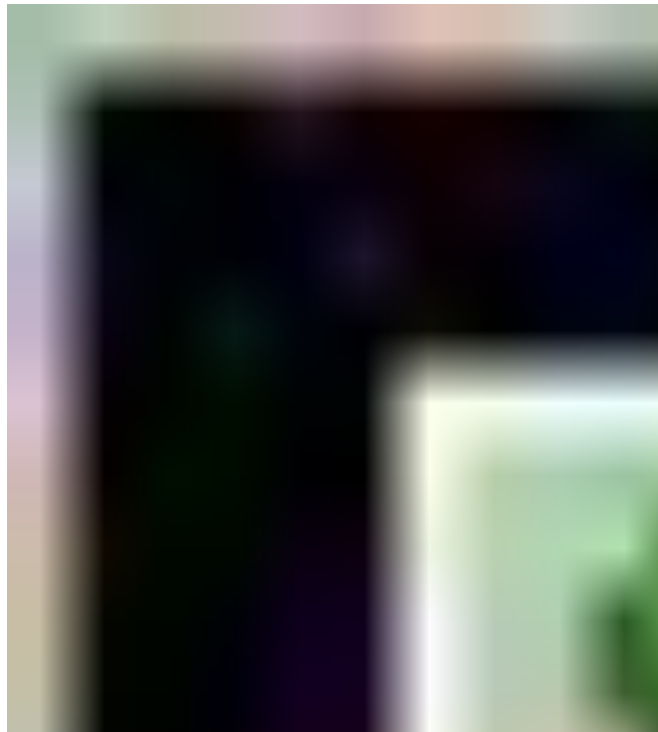
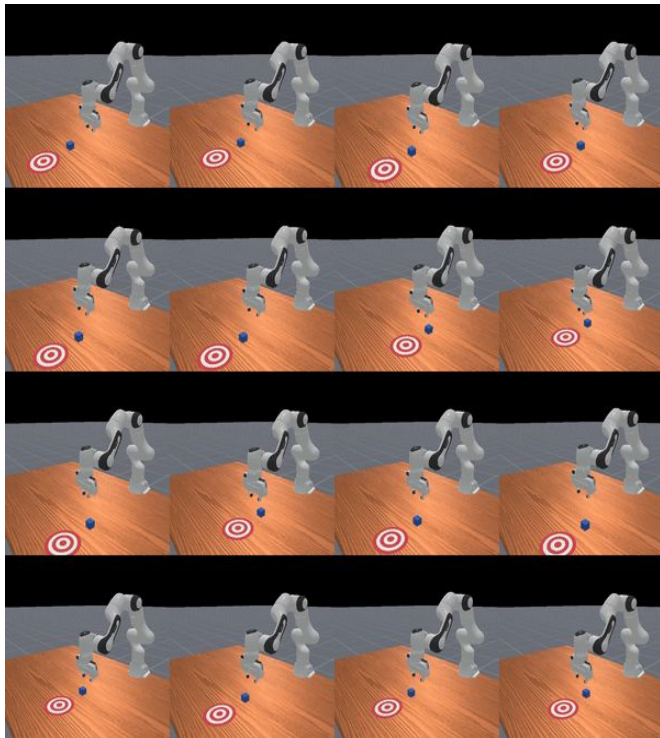
Ablation on caching



Online Trasformers for manipulation: our results



Online Trasformers for manipulation: our results



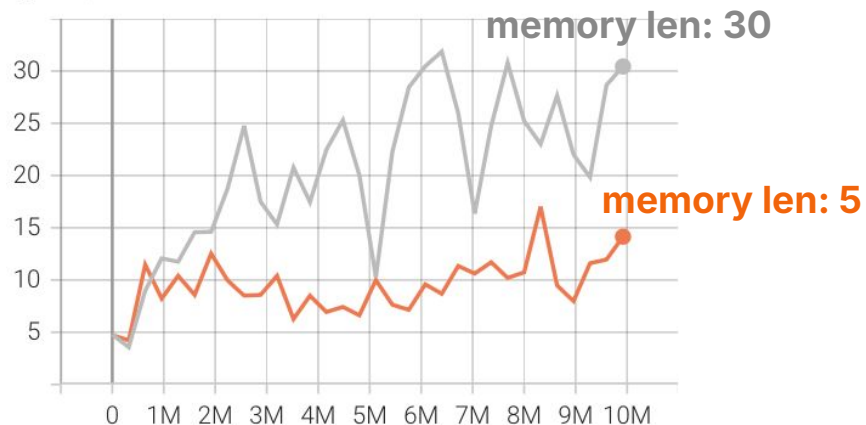
Online TransformerXL + PPO:

Results:

- Memory len **matters**
- Higher hidden dim leads to **worse training** (maybe change other parameters?)

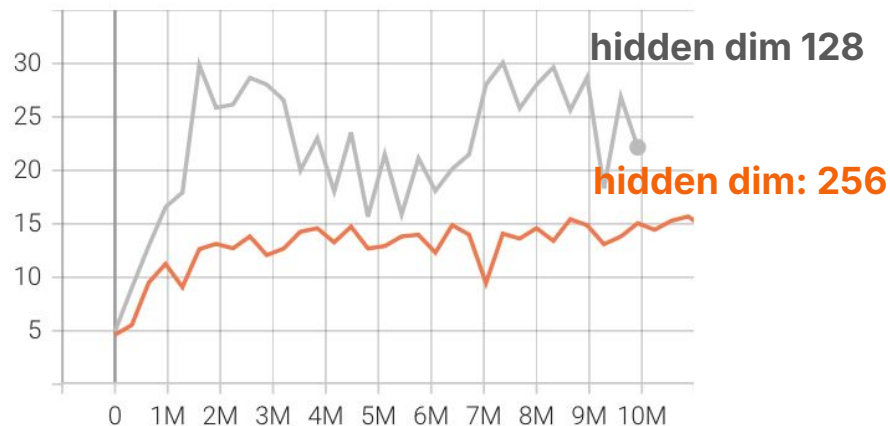
mem_len 30 vs mem_len 5

eval/return
tag: eval/return



transformer hidden dim 128 vs hidden dim 256

eval/return
tag: eval/return

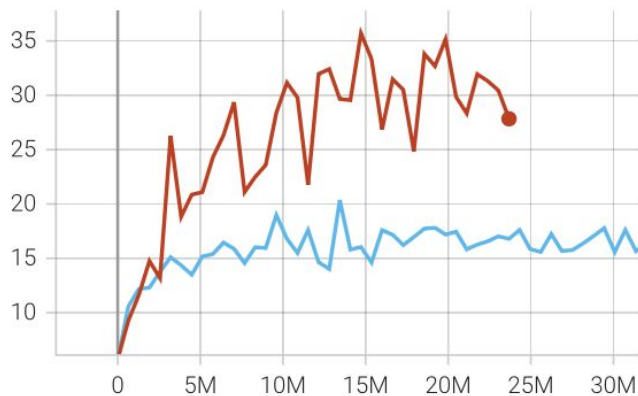


Online TransformerXL + PPO:

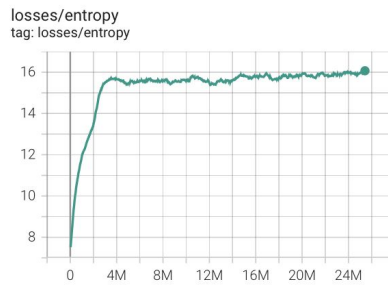
Results:

- Lower gamma
- Entropy loss with scheduling

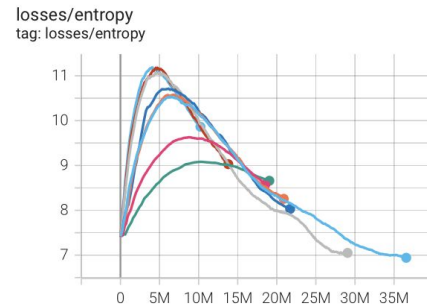
eval/return **gamma 0.6 vs 0.8**
tag: eval/return



before ent loss:

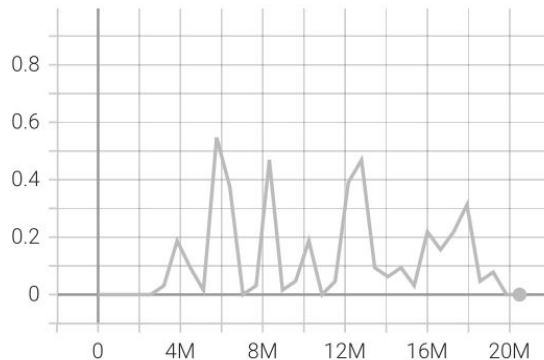


after ent loss:



eval/success_at_end
tag: eval/success_at_end

w.o. ent loss



Online TrasformerXL + PPO:

Results:

- Gating ablation study: **replace skip connection with GRU**

$$\overline{Y}^{(l)} = \text{RelativeMultiHeadAttention}(\text{LayerNorm}([\text{StopGrad}(M^{(l-1)}), E^{(l-1)}]))$$

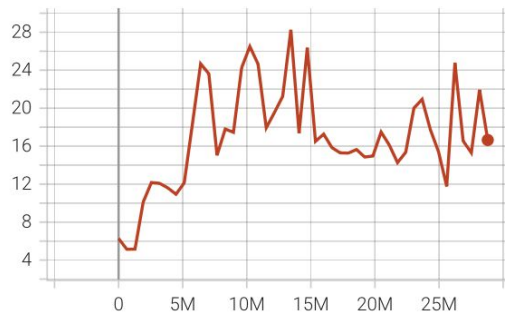
$$Y^{(l)} = g_{\text{MHA}}^{(l)}(E^{(l-1)}, \text{ReLU}(\overline{Y}^{(l)}))$$

$$\overline{E}^{(l)} = f^{(l)}(\text{LayerNorm}(Y^{(l)}))$$

$$E^{(l)} = g_{\text{MLP}}^{(l)}(Y^{(l)}, \text{ReLU}(\overline{E}^{(l)}))$$

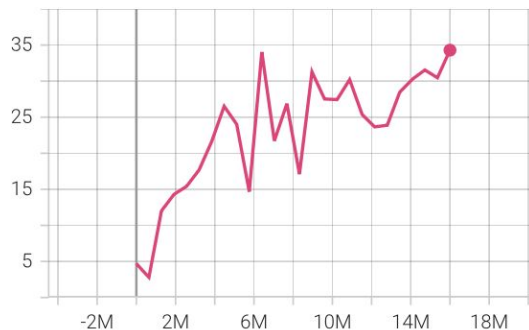
Without gating:

eval/return
tag: eval/return



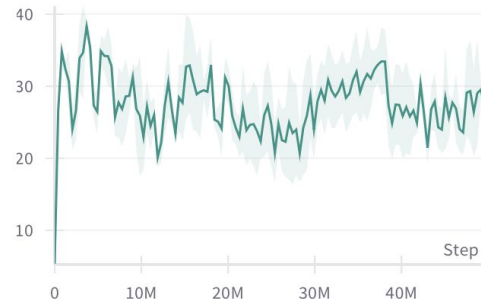
With 2 gatings:

eval/return
tag: eval/return



PPO-MLP

eval/return
eval_env_cfg.env_id: PushCube-v1



Online TransformerXL + PPO:

Results:

- With **shallow** action and value heads **training more stable**:

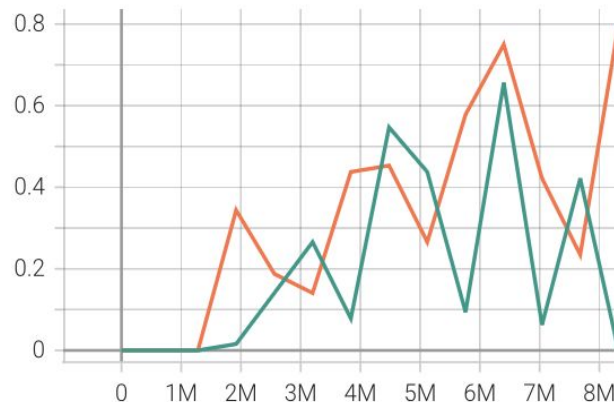
config 1: 2 linear layers

config 2: 4 linear layers

eval/return
tag: eval/return



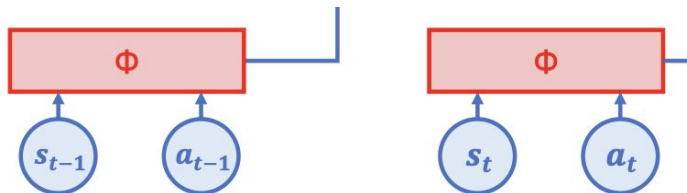
eval/success_at_end
tag: eval/success_at_end



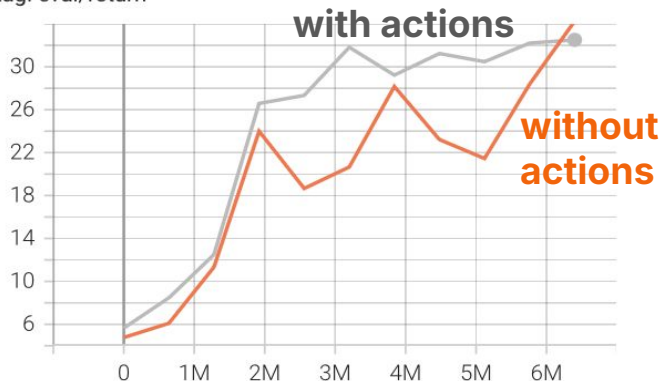
Online TransformerXL + PPO:

Results:

- Conditioning on actions stabilize training: embed obs, embed action → fuse in single embedding → store in memory



eval/return
tag: eval/return



eval/success_at_end
tag: eval/success_at_end

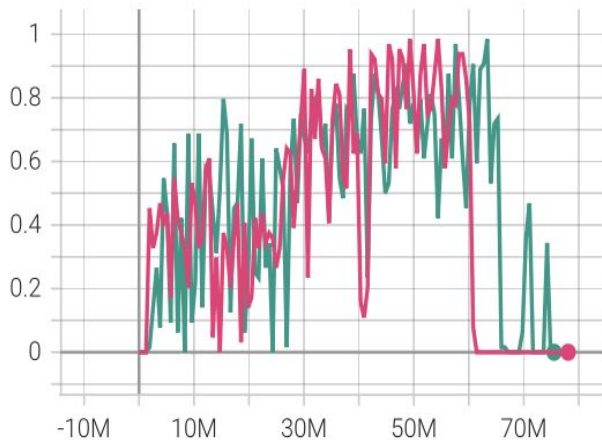


Online TrasformerXL + PPO:

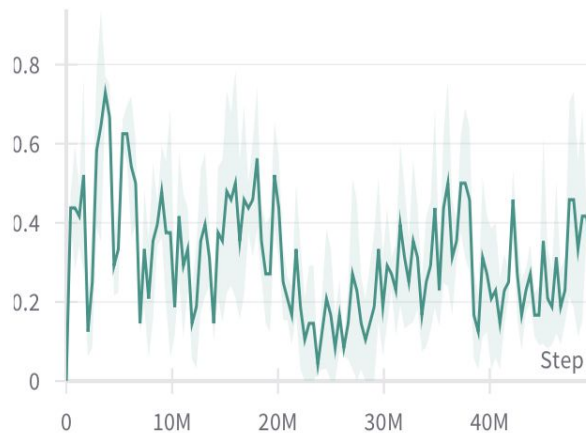
Results:

- Overall results

eval/success_at_end
tag: eval/success_at_end



eval/success_at_end
— eval_env_cfg.env_id: PushCube-v1



Thank you for your attention!